

S02- MultiLingual

url:

<https://api.groq.com/openai/v1/chat/completions>

Req Type: POST

```
{
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
  "messages": [
    {"role": "system", "content": "You are an API Specialist. Always respond in the same language as the user."},
    {"role": "user", "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்."}
  ],
  "temperature": 0.7,
  "max_completion_tokens": 1024
}

{
  "id": "chatcmpl-8c712373-311d-495f-a71e-d6ebbbceec243",
  "object": "chat.completion",
  "created": 1781892419,
  "model": "meta-llama/llama-4-scout-17b-16e-instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "நன்றி! உங்கள் மொழியில் விடை காண தயவு செய்யுங்கள். நான் அதற்கேற்ப பதில் அளிக்க முயற்சி செய்கிறேன்.\n\nஇந்த ஏபிஐஐ பயன்படுத்த, முதலில் ஒரு developer account ஐ உருவாக்குங்கள். பின், உங்கள் பயன்பாட்டிற்கு ஏற்ற endpoint ஐ தேர்ந்தெடுத்து, அதற்கான அணுகல் குறியீட்டைப் பெறுங்கள்.\n\nஅடுத்து, API க்கு அனுப்பும் HTTP வேண்டுகோளில் அந்த அணுகல் குறியீட்டை சேர்த்து, தேவையான தரவுகளை அனுப்புங்கள். API வேண்டுகோளுக்கு பதிலளிக்கும் போது, அது எந்த தரவுகளை திருப்பித் தருகிறது என்பதை சரிபார்த்து, அதன்படி உங்கள் பயன்பாட்டில் அந்த தரவுகளை பயன்படுத்துங்கள்.\n\nஎன்ன துல்லியமான வழிமுறைகள் வேண்டும்?"
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "queue_time": 0.159031513,
    "prompt_tokens": 71,
    "prompt_time": 0.002367437,
    "completion_tokens": 353,
    "completion_time": 0.799111798,
    "total_tokens": 424,
    "total_time": 0.801479235
  },
  "usage_breakdown": null,
  "system_fingerprint": "fp_f0bb21cb36",
  "x_groq": {
    "id": "req_01kvgh12pdeq98ra3w6ey0dfnp",
    "seed": 1682700475
  }
}
```

```

    },
    "service_tier": "on_demand"
}
=====
=====
With Temp:
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as
the user."
    },
    {
      "role": "user",
      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது
என்று சொல்லுங்கள்."
    }
  ],
  "temperature": 0.3,
  "max_completion_tokens": 1024
}
Response:
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as
the user."
    },
    {
      "role": "user",
      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது
என்று சொல்லுங்கள்."
    }
  ],
  "temperature": 0.3,
  "max_completion_tokens": 1024
}
=====
=====
S03-GenerateAPITestcasesFromSwaggerCode
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in
Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "I — INSTRUCTION\nGenerate Rest Assured Java code for Swagger
Petstore APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2)
Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4)
Use meaningful class, method, and variable names.\n5) Include imports required for Rest
Assured, TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD

```

operations for Pet resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nC — CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nP — PERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nO — OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside Swagger Petstore Pet API\n- Use business-readable method names\n\nTARGET API:\nSwagger Petstore\nResource: Pet\nOperations required:\n- Add a new pet\n- Find pet by ID\n- Update an existing pet\n- Delete a pet"

```
    }
  ]
}
```

```
java
import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
import static io.restassured.RestAssured.given;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.equalTo;
import static org.hamcrest.Matchers.notNullValue;
```

```
public class PetApiTest {
```

```
    private static final String BASE_URI = "https://petstore.swagger.io/v2";
    private static final String PET_RESOURCE = "/pet";
    private static int petId;
```

```
    @BeforeClass
    public void setup() {
        RestAssured.baseURI = BASE_URI;
    }
```

```
    @Test
    public void createPet() {
        Pet pet = new Pet();
        pet.setId(12345);
        pet.setName("testPet");
        pet.setStatus("available");

        Response response = given()
            .contentType(ContentType.JSON)
            .body(pet)
            .when()
            .post(PET_RESOURCE);

        petId = response.jsonPath().getInt("id");
        response.then()
            .statusCode(200)
            .body("name", equalTo("testPet"))
            .body("status", equalTo("available"));
    }
```

```
    @Test(dependsOnMethods = "createPet")
    public void getPetById() {
        Response response = given()
```

```

        .when()
        .get(PET_RESOURCE + "/" + petId);

    response.then()
        .statusCode(200)
        .body("id", equalTo(petId))
        .body("name", equalTo("testPet"))
        .body("status", equalTo("available"));
}

@Test(dependsOnMethods = "createPet")
public void updatePet() {
    Pet pet = new Pet();
    pet.setId(petId);
    pet.setName("updatedTestPet");
    pet.setStatus("pending");

    Response response = given()
        .contentType(ContentType.JSON)
        .body(pet)
        .when()
        .put(PET_RESOURCE);

    response.then()
        .statusCode(200)
        .body("name", equalTo("updatedTestPet"))
        .body("status", equalTo("pending"));
}

@Test(dependsOnMethods = "updatePet")
public void deletePet() {
    Response response = given()
        .when()
        .delete(PET_RESOURCE + "/" + petId);

    response.then()
        .statusCode(200);

    Response getResponse = given()
        .when()
        .get(PET_ResourceType + "/" + petId);

    getResponse.then()
        .statusCode(404);
}

private static class Pet {
    private int id;
    private String name;
    private String status;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {

```

```
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
}
```